

LAPORAN TEKNIS

Analisis Source Code SIKOS (Sistem Informasi Kos)

Dokumen analisis teknis terintegrasi ini mencakup identifikasi kelas, skema database (ERD), diagram arsitektur sistem, penjelasan komponen, serta alur request-response.

Backend: Laravel REST API (Sanctum) **Frontend:** React SPA (Vite + TypeScript)

Realtime: Node.js + Socket.IO Server

1 Ringkasan Sistem

SIKOS adalah aplikasi berbasis web untuk manajemen pemesanan kamar kos (Sistem Informasi Kos). Sistem ini menyediakan antarmuka bagi **Penghuni (Tenant)** untuk melihat kamar tersedia, melakukan pemesanan (booking), dan berkomunikasi secara real-time dengan pengelola kos (**Pak RT/Admin**). Pengelola memiliki panel dashboard khusus untuk melacak ketersediaan kamar, mengonfirmasi/membatalkan pesanan, dan membalas obrolan penghuni.

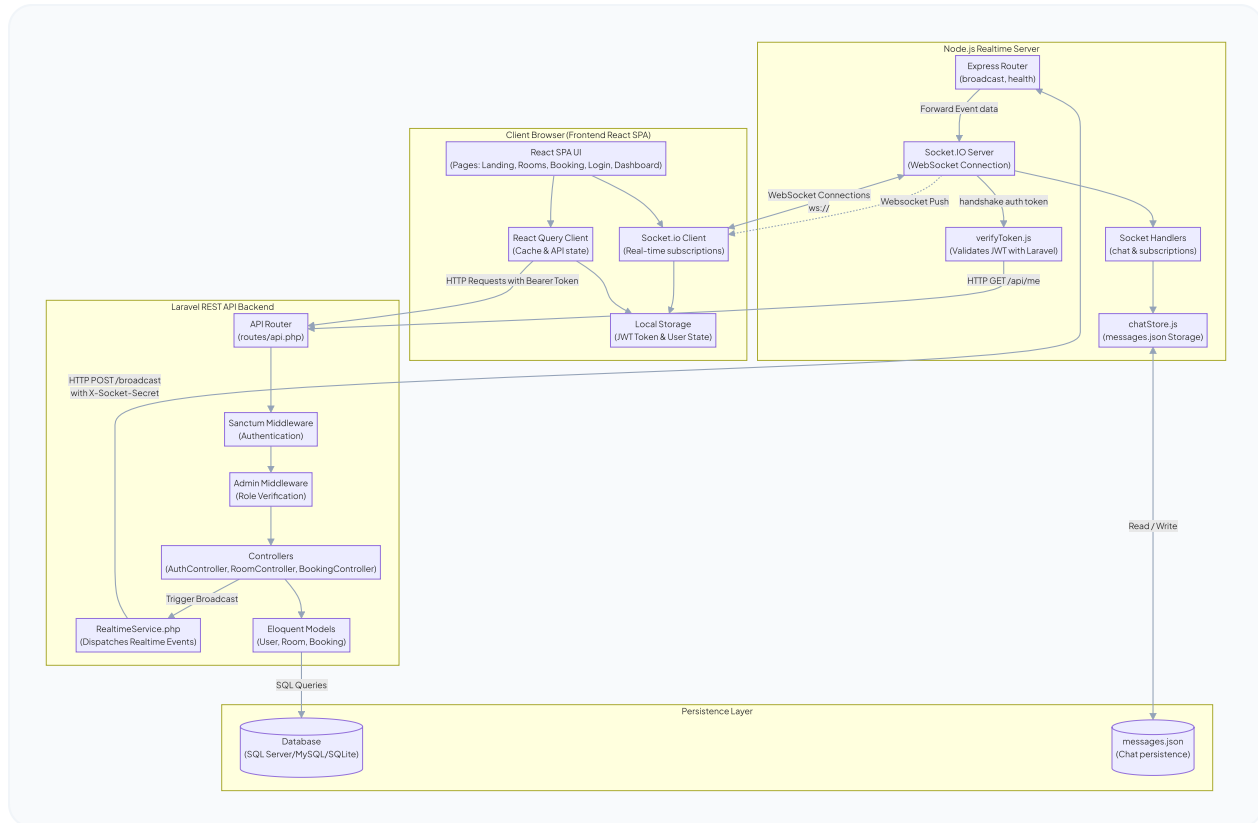
Aplikasi ini menggunakan model **decoupled architecture**:

- **Frontend** berkomunikasi dengan **Backend** melalui REST API JSON.
- **Frontend** terhubung ke **Realtime-Server** melalui WebSocket (Socket.IO).
- **Backend** mengirim event real-time (seperti booking baru atau perubahan status kamar) ke **Realtime-Server** menggunakan HTTP POST /broadcast yang dilindungi token rahasia, yang kemudian diteruskan ke klien yang berhak melalui Socket.IO.

- **Realtime-Server** memverifikasi token sesi klien dengan melakukan callback HTTP GET /api/me ke Laravel Backend.

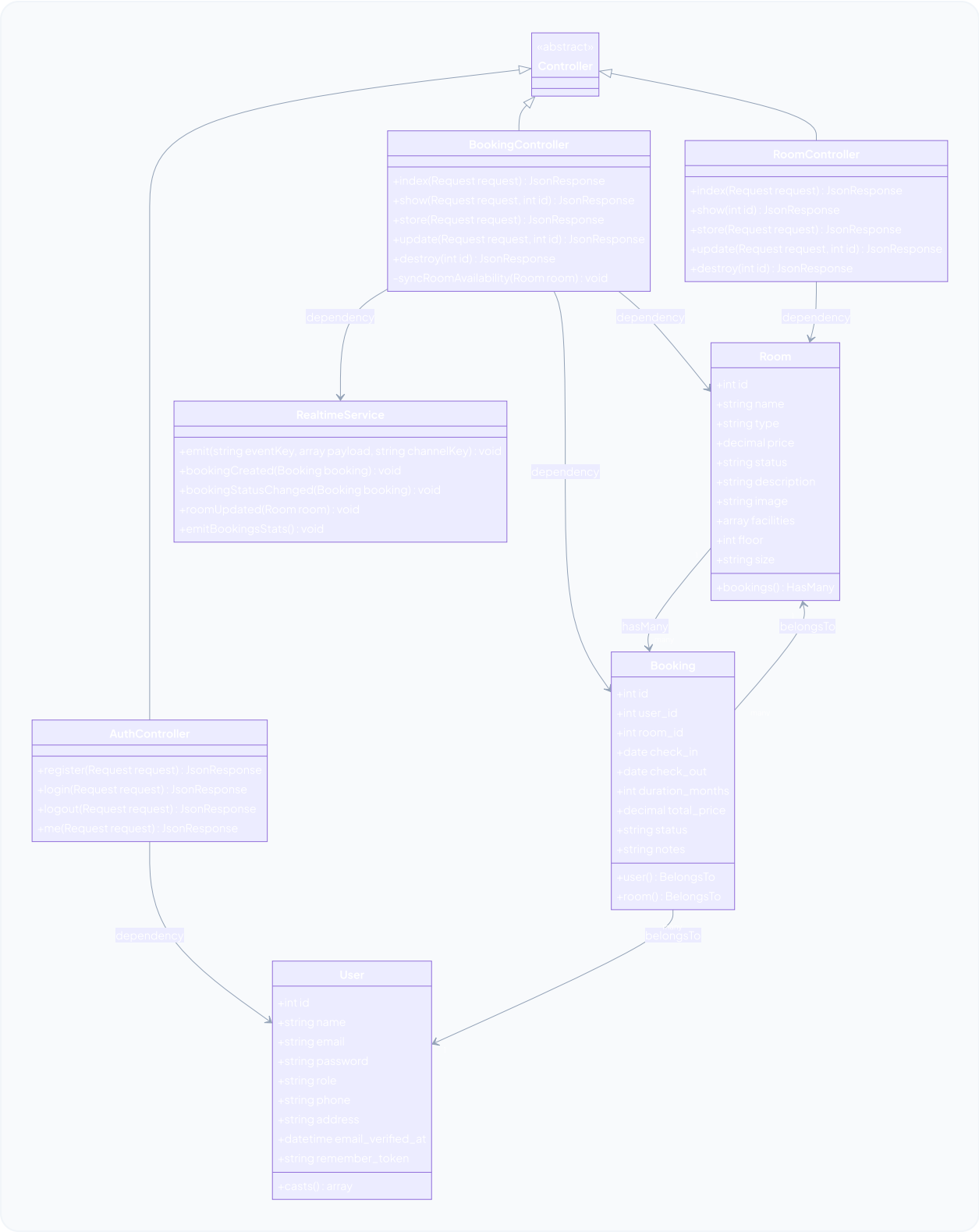
2 Diagram Arsitektur Sistem

Bagan di bawah menjelaskan alur request-response dan pertukaran data real-time antar subsistem:



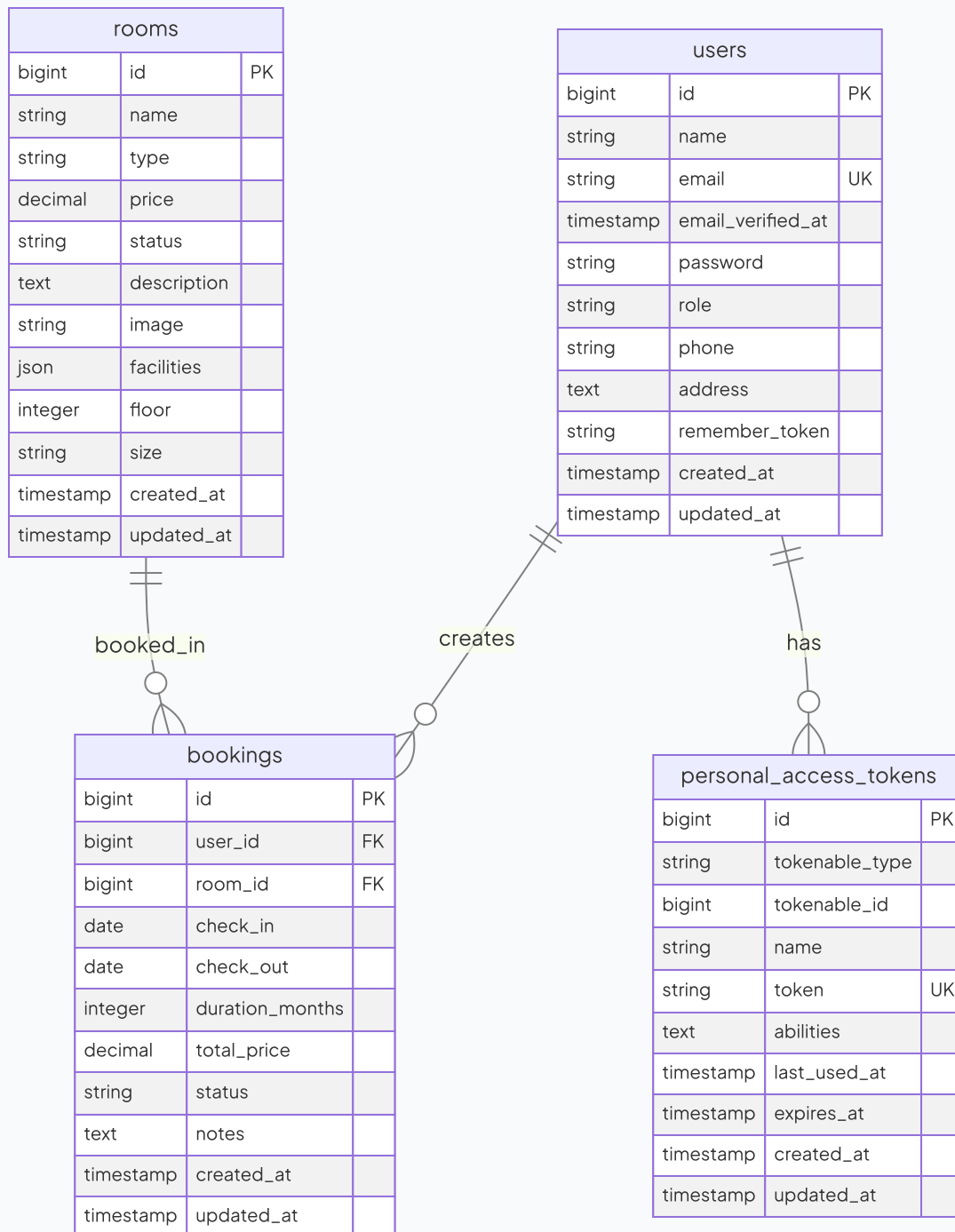
3 UML Class Diagram

Diagram kelas yang memetakan model, kontroler, dan ketergantungan relasi antar objek pada backend SIKOS:



4 Entity Relationship Diagram (ERD)

Skema basis data relasional lengkap beserta foreign key dan kardinalitas relasinya:



5 Penjelasan Komponen & Hubungan Sistem

A. Komponen Sistem

1. **Frontend (React SPA):** Terintegrasi dengan React Query untuk caching API state, menggunakan browser routing custom berbasis HTML5, serta mempertahankan konektivitas real-time lewat SocketContext.
2. **Backend (Laravel REST API):** Menyediakan persistensi data utama, otentikasi berbasis Sanctum Token, serta proteksi otorisasi peran via middleware `admin`.
3. **Realtime-Server (Node.js + Socket.IO):** Bertindak sebagai broker penyalur event real-time ke saluran `public`, `admin`, dan user privat. Riwayat chat disimpan secara lokal ke berkas `messages.json`.

B. Alur Data Utama

1. Alur Autentikasi & WebSocket:

User login via API → Laravel mengembalikan JWT/Sanctum Token → Token disimpan di `localStorage` → WebSocket client melakukan handshake dengan mengirim token → Node.js server memverifikasi token dengan memanggil endpoint API Laravel `GET /me` → Pengguna sukses terhubung ke channel WebSocket yang sesuai.

2. Alur Pembuatan Booking Baru:

Tenant mengisi form → `POST /api/bookings` ke Laravel → Validasi status kamar → Total harga dihitung → Data disimpan dengan status `pending` → Status kamar berubah menjadi `booked` → Laravel memicu `RealtimeService` mengirim event via `POST /broadcast` ke Node.js server → Node.js membroadcast event ke `admin` & `public` → UI `admin` dan `room` terupdate instan secara real-time.

6 Asumsi & Rekomendasi

- **Persistensi Obrolan:** Node.js server menyimpan obrolan dalam format berkas JSON lokal. Sebaiknya bermigrasi ke database NoSQL (seperti MongoDB/Redis) atau relational

database terpusat apabila ingin mengimplementasikan containerization (stateless container).

- **Sanctum Integration:** Verifikasi token Socket.IO dilakukan dengan melakukan request callback HTTP internal (/me) ke Laravel. Teknik ini sangat praktis, namun pastikan jaringan internal terisolasi dan cepat agar tidak menambah beban latency handshake.